

Capítulo 2: Aprendizaje Supervisado

Material del curso

Universidad de Salamanca

Facultades de Ciencias y Ciencias Químicas

24 de septiembre de 2026

- ❶ **2.1 Regresión:** predecir un valor continuo
 - Regresión lineal, mínimos cuadrados, descenso de gradiente
 - Regularización: *Ridge*, *Lasso*, *Elastic Net*
- ❷ **2.2 Clasificación:** predecir una categoría
 - Regresión logística, árboles, *Random Forest*, SVM
 - Ensamblados, redes neuronales y cómo evaluarlos

¿Qué es el aprendizaje supervisado?

- Tenemos **datos** (características de entrada, X) y **etiquetas** (respuestas correctas, y)
- El objetivo es aprender la relación $X \rightarrow y$
- Es como aprender con un “supervisor” que corrige tus respuestas durante el entrenamiento

Regresión (valor continuo)

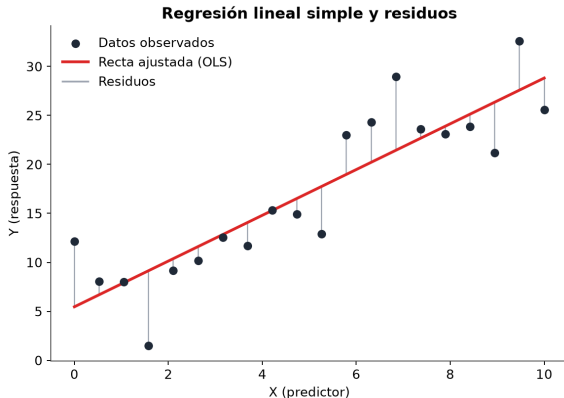
- Precio de una casa
- Temperatura de mañana

Clasificación (categoría)

- ¿Email es *spam*?
- ¿Imagen es gato, perro o pájaro?

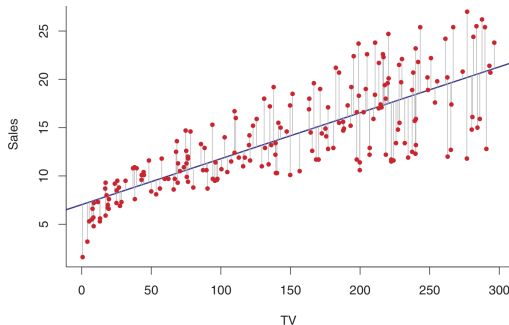
Regresión lineal

- **Simple:** un único predictor, $Y \approx \beta_0 + \beta_1 X$
- **Múltiple:** un coeficiente por cada característica, $Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \epsilon$
- β_0 (intercepto) y $\beta_1 \dots \beta_p$ (pesos) son lo que el modelo aprende de los datos
- En p dimensiones, el modelo representa un **hiperplano**



Ejemplo real: dataset *Advertising*

- Predice las ventas (*Sales*) a partir de la inversión en TV, radio y prensa
- La recta minimiza la suma de residuos al cuadrado (OLS)



Fuente: James, Witten, Hastie & Tibshirani (2013), *An Introduction to Statistical Learning*, Fig. 3.1. Libro de libre distribución (statlearning.com).

Mínimos cuadrados ordinarios (OLS)

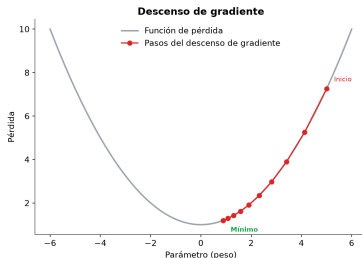
- Un **residual** $e_i = y_i - \hat{y}_i$ es la diferencia entre el valor real y el predicho
- OLS busca los pesos que **minimizan** la suma de los residuos al cuadrado:
$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$
- Tiene una **solución analítica exacta**: $\hat{\beta} = (X^T X)^{-1} X^T y$
- Interpretación geométrica: $\hat{y}$ es la proyección ortogonal de y sobre el espacio definido por X

Ejemplo: con solo 3 casas (m² y precio), OLS encuentra en un único paso la recta que mejor resume esa relación — sin necesidad de ensayo y error.

Descenso de gradiente

- Cuando los datos son masivos, resolver la ecuación exacta es demasiado costoso
- Algoritmo **iterativo**: parte de pesos aleatorios y los ajusta poco a poco en la dirección que más reduce el error
- $\theta \leftarrow \theta - \alpha \nabla J(\theta)$, donde α es la **tasa de aprendizaje**
- Variantes: *Batch* (todo el dataset), *Stochastic* (una muestra), *Mini-batch* (un grupo pequeño)

Ejemplo: es como bajar una montaña con niebla: en cada paso miras la pendiente bajo tus pies y avanzas hacia donde más desciende, hasta llegar al valle.

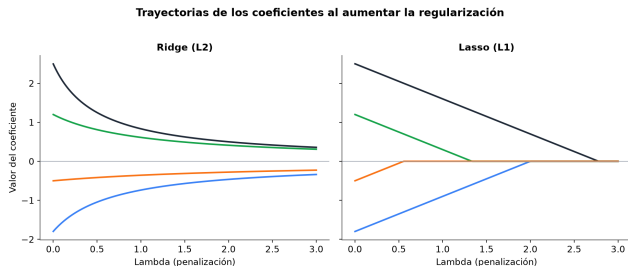


Regresión polinomial y el riesgo de sobreajuste

- Se añaden potencias de los predictores (X^2 , X^3 ...) para capturar curvas
- Sigue siendo un modelo **lineal en sus parámetros** — se puede seguir entrenando con OLS
- A mayor grado, mayor riesgo de memorizar el ruido (*overfitting*)

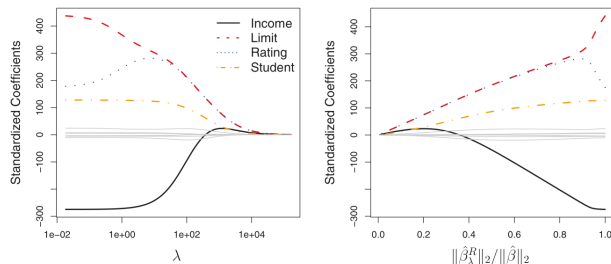
Regularización: *Ridge* y *Lasso*

- Penalizan los pesos grandes en la función de coste, forzando modelos más simples
- **Ridge** (L_2): penaliza $\sum \beta_j^2$ — encoge los coeficientes sin anularlos
- **Lasso** (L_1): penaliza $\sum |\beta_j|$ — puede llevar coeficientes exactamente a cero (selección de variables)
- **Elastic Net**: combina ambas penalizaciones



Ridge en datos reales: dataset *Credit*

- Al aumentar λ , los coeficientes se encogen hacia cero
- Reduce la varianza a cambio de un pequeño aumento del sesgo



Fuente: James, Witten, Hastie & Tibshirani (2013), *An Introduction to Statistical Learning*, Fig. 6.4.

Validación y diagnóstico en regresión

- **MSE / RMSE:** error cuadrático medio (y su raíz); penaliza fuerte los errores grandes
- **MAE:** error absoluto medio; más robusto ante valores atípicos
- R^2 : proporción de varianza explicada por el modelo (1 = perfecto, 0 = tan bueno como predecir la media)
- **Curvas de validación:** graficar error de entrenamiento y validación frente a la complejidad revela si hay sesgo alto (ambos errores altos) o varianza alta (brecha entre ambos)

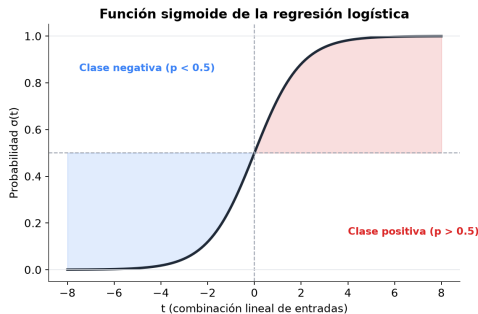
- **Regresión lineal**: modelo simple pero muy potente
- **OLS**: solución exacta; **descenso de gradiente**: método general y escalable
- **Regresión polinomial**: captura relaciones no lineales, a costa de más riesgo de *overfitting*
- **Regularización**: *Ridge* encoge, *Lasso* selecciona variables

Clasificación: predecir categorías

- El objetivo es predecir una **categoría**, no un número continuo
- Ejemplos: *spam* vs. *no-spam*, enfermo vs. sano, gato vs. perro vs. pájaro
- La **regresión logística** empieza estimando la **probabilidad** de pertenecer a una clase

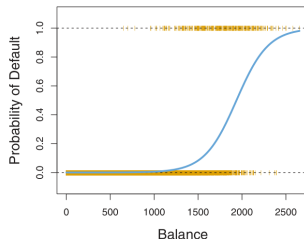
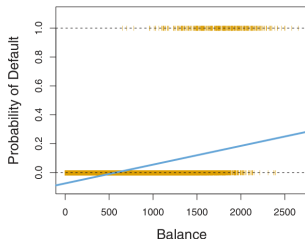
La función sigmoide

- Transforma cualquier valor real en una probabilidad entre 0 y 1
- $\sigma(t) = \frac{1}{1 + e^{-t}}$, donde t es el *logit* (combinación lineal de las *features*)
- Se entrena minimizando la **entropía cruzada** (*log loss*) con descenso de gradiente



¿Por qué no usar regresión lineal?

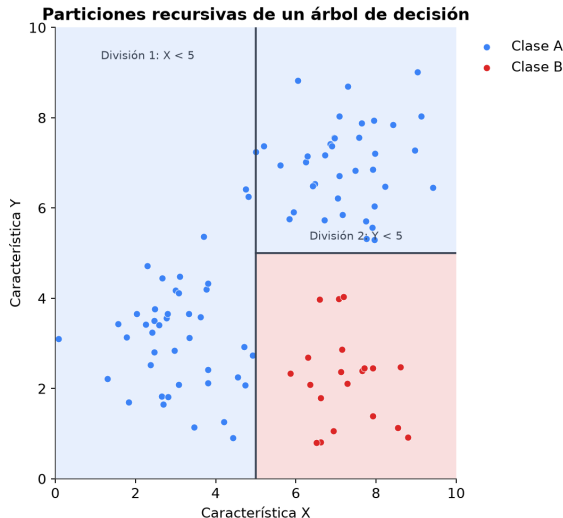
- La regresión lineal puede predecir probabilidades < 0 o > 1 (sin sentido)
- La logística usa la sigmoide para mantener la salida siempre en $[0, 1]$



Fuente: James, Witten, Hastie & Tibshirani (2013), *An Introduction to Statistical Learning*, Fig. 4.2.

Árboles de decisión

- Dividen el espacio de *features* con cortes sucesivos ortogonales, como un diagrama de flujo de preguntas “sí/no”
- Algoritmo **CART**: cada división busca maximizar la pureza de los nodos hijos (índice Gini o entropía)
- Fáciles de interpretar, pero propensos a *overfitting* sin **poda**

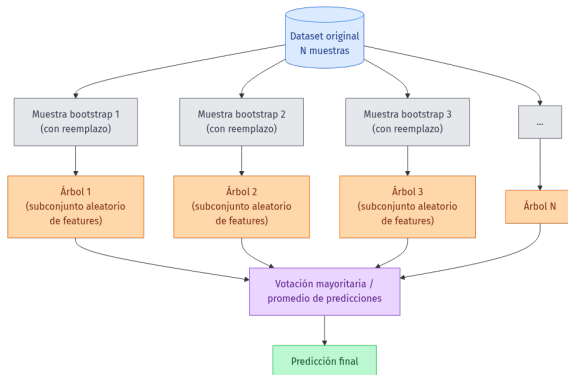


Ejemplo: “¿el email contiene la palabra ‘gratis’? → sí → ¿tiene más de 3 enlaces? → sí → *spam*”.

Bosques aleatorios (*Random Forests*)

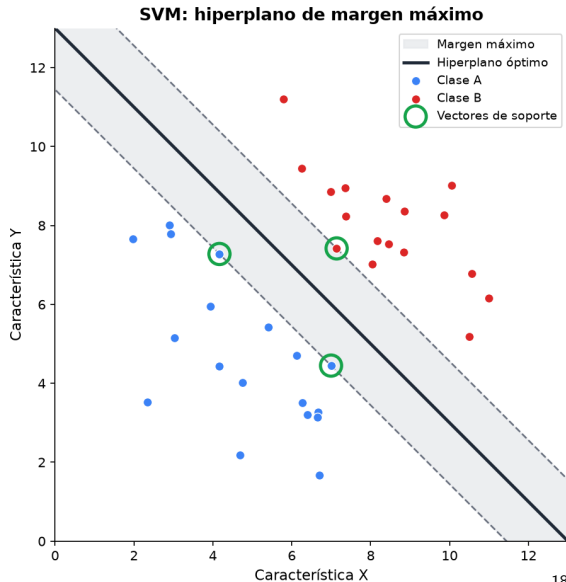
- Combinan **muchos árboles** para reducir la varianza y mejorar la robustez
- **Bagging**: cada árbol se entrena con una muestra *bootstrap* distinta (con reemplazo)
- **Decorrelación**: en cada división solo se considera un subconjunto aleatorio de *features*

Varias muestras *bootstrap*, un árbol distinto por cada una, y las predicciones se combinan por votación mayoritaria o promedio.



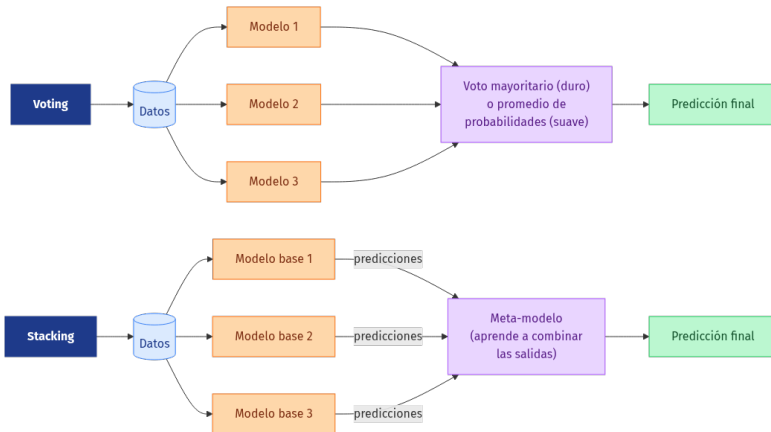
SVM: margen máximo

- Busca el **hiperplano** que separa las clases con el **margen máximo**
- Los **vectores de soporte** son los únicos puntos que determinan la frontera
- El *kernel trick* permite fronteras no lineales, mapeando los datos a un espacio de más dimensiones

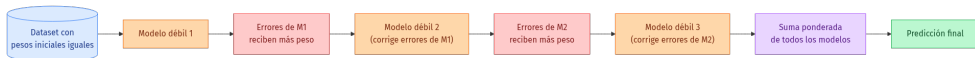


Voting y stacking

- Combinan **varios modelos** para lograr una predicción mejor que cualquiera por separado
- **Voting**: voto mayoritario o promedio de probabilidades, con una regla fija
- **Stacking**: un meta-modelo **aprende** cómo combinar las salidas de los modelos base



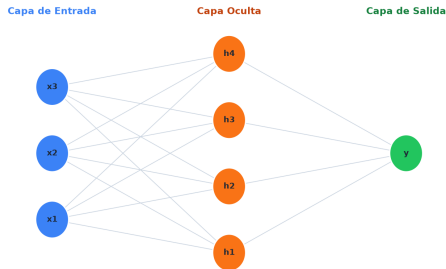
- A diferencia del entrenamiento en paralelo de los bosques aleatorios, el *boosting* entrena modelos de forma **secuencial**
- Cada nuevo modelo intenta corregir los errores del anterior, dando más peso a las instancias mal clasificadas
- Ejemplos modernos: *XGBoost*, *Gradient Boosting*



Redes neuronales

- Capas de neuronas conectadas por pesos ajustables
- Cada neurona aplica una función de activación no lineal: **ReLU** en capas ocultas, **Softmax** en la capa de salida
- **Retropropagación** (*backpropagation*): propaga el error desde la salida hacia atrás para ajustar todos los pesos

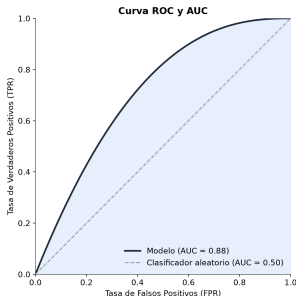
Arquitectura de una red neuronal totalmente conectada



Evaluación de clasificadores

- **Exactitud** (*accuracy*): puede engañar si las clases están desbalanceadas
- **Precisión** y **recall**: qué tan fiables son los positivos vs. cuántos positivos reales se encuentran
- **F1**: equilibrio entre ambas
- **ROC-AUC**: compara modelos sin depender de un umbral fijo

Ejemplo: si el 99% de las transacciones no son fraude, un modelo que siempre diga “no es fraude” tendría 99% de exactitud sin haber aprendido nada útil.



- **Superar una línea de base:** antes de complicarse, comprobar que se le gana a un clasificador trivial (aleatorio)
- **Curvas de aprendizaje:** pérdida y exactitud en entrenamiento/validación revelan el punto exacto de sobreajuste
- ***Early stopping*:** detener el entrenamiento cuando la validación deja de mejorar
- **Análisis de errores:** revisar a mano los casos donde el modelo falla

- **Regresión logística:** extiende la regresión lineal a probabilidades
- **Árboles:** interpretables, propensos a *overfitting*; **Random Forest:** lo corrige combinando muchos
- **SVM:** separación óptima, con *kernel trick* para fronteras complejas
- **Ensamblados** (*voting, stacking, boosting*): combinan modelos para mejorar
- **Redes neuronales:** potentes pero más complejas de interpretar

Fin del Capítulo 2

Siguiente: Capítulo 3 — Aprendizaje No Supervisado